

AWS Jam 实战挑战总结

Lambda Layers 共享代码库

服务: AWS Lambda | 难度: 入门 ~ 中级

挑战概览

本次 AWS Jam 挑战围绕 AWS Lambda Layers（层）展开，分为两个任务：

1. **Task 1:** 为两个 Lambda 函数添加共享代码层，解决 `Runtime.ImportModuleError` 错误。
2. **Task 2:** 发布新版本的 Layer，并将其中一个函数升级到新版本。

涉及资源：

- 共享库: `tattooine-common`（存储于 Amazon S3）
- 函数一: `LabStack-prewarm-{random}-ChallengeFunctionOne-{random}`
- 函数二: `LabStack-prewarm-{random}-ChallengeFunctionTwo-{random}`
- 兼容架构: `x86_64`
- 兼容运行时: `Node.js 22.x`（Task 2 额外支持 `Node.js 18.x`）

Task 1: 创建 Lambda Layer 并关联函数

背景与问题

两个 Lambda 函数在部署时未包含依赖库 `tattooine-common`，导致运行时抛出如下错误：

```
Runtime.ImportModuleError: Error: Cannot find module 'tattooine-common'
```

库文件已上传至 S3，路径为：

```
s3://aws-jam-challenge-resources-{REGION}/lambda-layers-shared-code/task-one/layer-task-one.zip
```

解决方案：使用 Lambda Layers

Lambda Layers 是 Lambda 提供的共享机制，用于在多个函数间复用依赖库、自定义运行时或配置文件。实际操作时，可在函数详情页的 **Layers** 区域进行关联。

操作步骤

步骤一：创建 Lambda Layer

1. 进入 AWS Lambda 控制台，点击左侧菜单 **Layers**
2. 点击右上角 **Create layer**
3. 填写配置：
 - **Name:** tattooine-common
 - **Upload method:** 选择 *Upload a file from Amazon S3*
 - **S3 link URL:** 填入上方 S3 路径（替换 {REGION}）
 - **Compatible architectures:** 勾选 x86_64
 - **Compatible runtimes:** 选择 Node.js 22.x
4. 点击 **Create**，完成 Layer **Version 1** 的创建

步骤二：将 Layer 关联到两个函数（分别对 FunctionOne 和 FunctionTwo 执行）

1. 进入 **Lambda** → **Functions**，打开目标函数
2. 向下滚动至 **Layers** 区域，点击 **Add a layer**
3. 选择 **Custom layers** → 选择 tattooine-common → 选择 **Version 1**
4. 点击 **Add** 保存

原理说明

Layer 被关联后，Lambda 执行环境会自动将其内容解压至 `/opt/` 目录：

```
/opt/  
  nodejs/  
    node_modules/  
      tattooine-common/ ← 函数可直接 import
```

函数代码无需修改，`require('tattooine-common')` 即可正常导入。

Task 2: 发布 Layer 新版本并灰度升级

背景与问题

tattooine-common 发布了新版本，新库已上传至 S3:

```
s3://aws-jam-challenge-resources-{REGION}/lambda-layers-shared-code/task-two/layer-task-two.zip
```

需要先在 一个函数上验证新版本，另一个函数暂不升级。

核心概念: Layer 版本不可变性

关键知识点: Lambda Layer 的每个版本是不可变的 (Immutable)。一旦创建，无法修改其内容。要更新 Layer，必须发布一个新版本，每次发布版本号自动递增。

操作步骤

步骤一: 在已有 Layer 上发布新版本

1. 进入 Lambda 控制台 → **Layers**, 找到 tattooine-common
2. 点击进入该 Layer 详情页
3. 点击右上角 **Create version**
4. 填写配置:
 - **Description:** v2 - new release (可选)
 - **S3 link URL:** 填入 task-two 的 S3 路径
 - **Compatible architectures:** x86_64
 - **Compatible runtimes:** Node.js 22.x 和 Node.js 18.x

5. 点击 **Create**, 生成 **Version 2**

步骤二: 更新 FunctionOne 到 Version 2

1. 打开 ChallengeFunctionOne, 进入 **Layers** 区域
2. 点击 **Edit**, 先删除已关联的 Version 1 (点击旁边的删除按钮)
3. 再点击 **Add a layer** → Custom layers → tattooine-common → **Version 2**
4. 点击 **Save**

注意：同一个 Layer 的不同版本**不能同时**关联到同一个函数。若不先移除旧版本，直接添加新版本会报错：

Two different versions of the same layer are not allowed to be referenced in the same function.

正确做法是：先删除旧版本，再添加新版本。

FunctionTwo 保持关联 Version 1，实现灰度验证。

核心知识点汇总

什么是 Lambda Layers?

Lambda Layers 是一种将**依赖库、自定义运行时、配置或数据文件**与函数代码分离的机制：

- 避免每个函数重复打包相同依赖，减小部署包体积
- 多个函数可共享同一个 Layer，统一管理依赖版本
- 最多可为一个函数关联 **5 个** Layer
- 所有 Layer 内容合并后解压到执行环境的 `/opt/` 路径下
- Layer 仅适用于 **.zip 包部署** 的 Lambda 函数；容器镜像函数不使用 Layer 机制

Layer 版本管理

特性	说明
版本不可变	已发布的版本内容无法修改，只能新建版本
版本号递增	每次 Create version，版本号自动 +1（如 v1 → v2 → v3）
ARN 唯一标识	每个版本有独立的 ARN，格式为 <code>arn:aws:lambda:{region}:{account}:layer:{name}:{version}</code>
函数固定版本	函数不会自动跟随 Layer 更新，需手动修改关联的版本号
旧版本不删除	发布新版本后旧版本继续存在，不影响关联了旧版本的函数

Layer 目录结构 (Node.js)

```
layer.zip
  nodejs/
    node_modules/
      your-library/
        index.js
        package.json
```

Lambda 会将此结构解压到 `/opt/`。对于 Node.js，常见可用路径包括：

- `/opt/nodejs/node_modules`
- `/opt/nodejs/node18/node_modules`
- `/opt/nodejs/node20/node_modules`
- `/opt/nodejs/node22/node_modules`

因此，通用的 `nodejs/node_modules` 结构通常最方便；若需要按运行时版本细分，也可使用对应的 `nodeXX` 子目录。

灰度发布策略

通过 Layer 版本可实现分批升级：

1. 发布新 Layer 版本
2. 仅将测试函数切换到新版本
3. 验证无误后，再逐步更新其他函数

这与软件发布中的**金丝雀发布 (Canary Release)** 思路一致。

错误排查速查

错误信息	原因与解决方法
<code>Runtime.ImportModuleError</code>	函数缺少依赖库。创建 Lambda Layer 并关联到函数。
<code>Two different versions of the same layer are not allowed...</code>	同一 Layer 的两个版本同时关联到同一函数。先删除旧版本，再添加新版本。
<code>Layer content is invalid</code>	Layer zip 包目录结构不符合规范。检查 <code>nodejs/node_modules/</code> 路径是否正确。

本文档整理自 AWS Jam 实战挑战 **Lambda Layers Shared Code** 系列题目 | 编写
于 2026 年 3 月 9 日